Aufgabenblatt 1

1. Elben und Orks

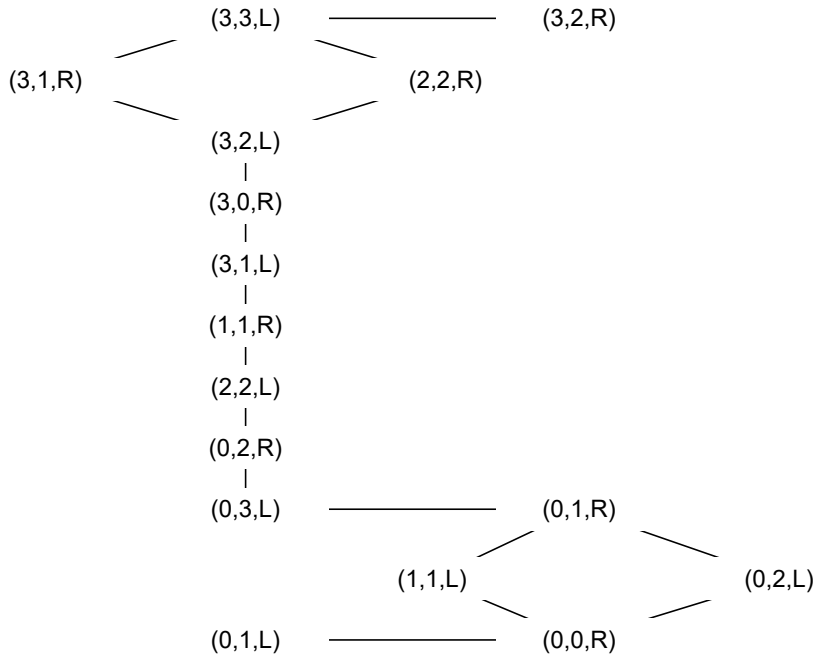
Ich gehe davon aus, dass ein Ufer ohne Elben sicher ist. Sonst wäre die Aufgabe nicht lösbar.

Ein Zustand ist (e, o, p). Dabei sind e und o die Anzahl der Elben und Orks links. p ist die Position des Pferdes (L oder R). Rechts befinden sich 3 − e Elben und 3 − o Orks. Ein Zustand ist erlaubt, wenn an beiden Ufern entweder keine Elben sind oder mindestens genauso viele Elben wie Orks.

Startzustand: (3, 3, L). Zielzustand: (0, 0, R).

Mögliche Aktionen sind E, EE, O, OO und EO. Die Mitreisenden müssen am Ufer des Pferdes sein. Bei einer Fahrt nach rechts werden ihre Anzahlen links abgezogen, bei einer Rückfahrt addiert. Das Pferd wechselt das Ufer; der Folgezustand muss erlaubt sein.

Im Problemgraphen sind Zustände die Knoten und erlaubte Überquerungen die Kanten. Die Skizze zeigt alle erreichbaren Zustände. Jede Kante ist in beide Richtungen möglich.



Eine mögliche Lösung (E = Elb, O = Ork):

OO →, O ←, OO →, O ←, EE →, EO ←, EE →, O ←, OO →, O ←, OO →.

2. Suchverfahren

Ich besuche Nachfolger alphabetisch. Die Zielprüfung erfolgt beim Entnehmen; die Zielentnahme zählt als Schleifendurchlauf. Bei Graph Search werden bereits entdeckte Städte nicht erneut aufgenommen. A* verhindert nur Zyklen innerhalb des jeweiligen Pfades. Gezählt werden die offenen Einträge.

Tiefensuche

Entnahmereihenfolge: Würzburg, Erfurt, Frankfurt, Kassel, München.

Weg: Würzburg - Frankfurt - Kassel - München.
Kosten: 217 + 173 + 502 = 892 km.
Maximal 3 Einträge im Stapel, 5 Schleifendurchläufe.

Breitensuche

Entnahmereihenfolge: Würzburg, Erfurt, Frankfurt, Nürnberg, Kassel, Mannheim, München.

Weg: Würzburg - Nürnberg - München.
Kosten: 103 + 167 = 270 km.
Maximal 4 Einträge in der Warteschlange, 7 Schleifendurchläufe.

A* mit den vorgegebenen Schätzungen

A* wählt jeweils den Eintrag mit dem kleinsten Wert f(n) = g(n) + h(n).

Entnahmereihenfolge: Würzburg, Frankfurt, Mannheim, Karlsruhe, Erfurt, Augsburg, Nürnberg, München.
Die f-Werte sind: 170, 317, 502, 392, 586, 632, 640, 270.

Weg: Würzburg - Nürnberg - München. Kosten: 270 km.
Maximal 4 offene Einträge, 8 Schleifendurchläufe.

Prüfung und Korrektur der Schätzungen

Eine zulässige Heuristik darf die tatsächlichen kürzesten Restkosten nicht überschätzen. Für Nürnberg werden 537 km geschätzt, obwohl München direkt über eine Kante mit 167 km erreichbar ist. Die vorgegebene Heuristik ist daher nicht zulässig; eine Optimalitätsgarantie besteht damit nicht. Alle anderen Schätzungen sind zulässig.

Ich korrigiere h(Nürnberg) auf 167 km. Alle anderen Werte bleiben gleich.

Neue Entnahmereihenfolge: Würzburg, Nürnberg, München.
Die f-Werte sind: 170, 270, 270.
Weg: Würzburg - Nürnberg - München. Kosten: 270 km.
Maximal 4 offene Einträge, 3 Schleifendurchläufe.

Die Tiefensuche liefert hier den längsten Weg. Die Breitensuche minimiert die Anzahl der Kanten. A* mit der korrigierten Heuristik findet den günstigsten Weg und benötigt hier die wenigsten Schleifendurchläufe.

3. Dominanz

Eine zulässige Heuristik h1 dominiert eine zulässige Heuristik h2, wenn für jeden Zustand n gilt:

$h1(n) \geq h2(n)$.

h1 schätzt die Restkosten damit mindestens genauso genau ein, ohne sie zu überschätzen. A* untersucht dadurch in der Regel weniger Knoten. Genauer: Es werden nicht mehr Knoten mit $f < C^*$ expandiert. Bei $f = C^*$ hängt die genaue Anzahl auch von der Reihenfolge bei Gleichstand ab. Die Optimalität bleibt erhalten.

Beispiel 1: Bei der Routenplanung dominiert die Luftlinienentfernung die Heuristik $h(n) = 0$, sofern die Luftlinie eine zulässige Schätzung ist.

Beispiel 2: In einem Raster mit Bewegung nach oben, unten, links und rechts und Kosten 1 pro Schritt dominiert die Manhattan-Distanz die euklidische Distanz. Bei drei Feldern horizontal und vier Feldern vertikal ergeben sich 7 statt 5. Tatsächlich sind mindestens 7 Schritte nötig.

4. Optimalität von A*

Seien C^* die Kosten einer optimalen Lösung. Die Heuristik sei zulässig und für Zielknoten gelte $h = 0$. Das Ziel wird beim Entnehmen geprüft.

Angenommen, A* entnimmt zuerst ein suboptimales Ziel Z. Dann gilt:

$f(Z) = g(Z) > C^*$.

In der offenen Datenstruktur liegt aber noch ein Knoten n auf einem optimalen Lösungspfad. Da die Heuristik die Restkosten nicht überschätzt, gilt:

$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*$.

Damit ist $f(n)$ kleiner als $f(Z)$. A* müsste also n vor Z entnehmen, weil immer der kleinste f-Wert gewählt wird. Das widerspricht der Annahme.

Die zuerst von A* zurückgegebene Lösung ist deshalb optimal.